

Rapport d'Architecture de Sécurité et d'Audit : Application "Kechbus Ticket"

Introduction

Ce rapport détaille la stratégie de cybersécurité conçue pour l'application **Kechbus Ticket** suite à la phase de développement. L'objectif de ce document est de définir les protocoles stricts de sécurisation des journaux d'événements (logs), d'établir une architecture cloud résiliente, et de présenter la méthodologie ainsi que les outils requis pour une analyse approfondie des vulnérabilités de l'application.

1. Stratégie de Gestion et de Sécurisation des Journaux (Logs)

Les journaux d'événements constituent la principale source de vérité en cas d'incident de sécurité. En cas de compromission, la première action d'un attaquant consiste généralement à altérer ces logs pour effacer ses traces. La politique suivante a donc été établie :

- **Centralisation** : Les logs ne sont pas stockés localement sur les serveurs de l'application. Ils sont transférés en temps réel vers un serveur de journalisation centralisé et isolé.
- **Chiffrement de bout en bout** : * *En transit* : Utilisation des protocoles TLS 1.2 ou 1.3 pour garantir la confidentialité des logs lors de leur transfert.
 - *Au repos* : Chiffrement des volumes de stockage de logs via des algorithmes robustes (ex. AES-256).
- **Immuabilité (WORM)** : Le stockage est configuré en mode "Write-Once-Read-Many" (Écriture unique, lectures multiples). Tout log enregistré ne peut être ni modifié ni supprimé, même par un compte administrateur, pendant la durée de rétention légale et technique définie.
- **Contrôle d'accès strict (RBAC)** : L'accès aux logs est régi par un contrôle basé sur les rôles. Seul le personnel de sécurité et d'audit dispose des privilèges de lecture.
- **Minimisation des données (Privacy by Design)** : Des filtres sont appliqués pour garantir qu'aucune donnée utilisateur sensible (mots de passe en clair, données bancaires, jetons de session) n'est inscrite dans les logs, évitant ainsi que ces derniers ne deviennent un vecteur de fuite d'informations.

2. Architecture de Sécurité Cloud

Le déploiement de Kechbus Ticket dans un environnement cloud repose sur un modèle de responsabilité partagée. L'architecture de sécurité mise en place s'articule autour des axes suivants :

- **Gestion des Identités et des Accès (IAM) :** Application stricte du principe de moindre privilège (PoLP). Les services, instances et développeurs ne disposent que des droits strictement nécessaires à leur fonction. L'authentification multifacteur (MFA) est imposée pour tout accès administratif.
- **Segmentation et Sécurité Réseau :** L'architecture réseau isole la base de données et les services backend au sein d'un sous-réseau privé, inaccessible directement depuis l'Internet public. Seuls les composants frontaux (équilibres de charge, passerelles API) sont exposés dans le sous-réseau public.
- **Pare-feu d'Application Web (WAF) :** Déploiement d'un WAF en périphérie pour inspecter le trafic entrant. Ce dispositif bloque les attaques courantes de couche applicative (couche 7 du modèle OSI), telles que les injections SQL (SQLi) ou le Cross-Site Scripting (XSS), avant qu'elles n'atteignent le backend.
- **Gestion centralisée des secrets :** Les identifiants de base de données et les clés d'API (notamment pour les paiements) ne sont jamais codés en dur. L'application s'appuie sur un gestionnaire de secrets dédié (tel que AWS Secrets Manager ou HashiCorp Vault) pour injecter ces variables de manière sécurisée lors de l'exécution.

3. Méthodologie d'Analyse des Vulnérabilités et Outillage

Afin d'assurer la robustesse de l'application Kechbus Ticket, l'évaluation de la sécurité est réalisée selon une approche multidimensionnelle, nécessitant l'intégration de plusieurs outils spécialisés :

- **Tests Statiques de Sécurité des Applications (SAST) :**
 - *Objectif :* Analyser le code source à la recherche de failles logiques, de mauvaises pratiques ou de secrets codés en dur avant la phase de compilation.
 - *Outils recommandés :* **SonarQube** ou **Snyk Code**.
- **Tests Dynamiques de Sécurité des Applications (DAST) :**
 - *Objectif :* Interagir avec l'application web en cours d'exécution depuis l'extérieur. Cette méthode simule le comportement d'un attaquant pour

identifier les vulnérabilités exploitables (XSS, SQLi, erreurs de configuration).

- *Outils recommandés* : **OWASP ZAP** (solution open-source) ou **Burp Suite Professional** (standard de l'industrie).
- **Analyse de la Composition Logicielle (SCA)** :
 - *Objectif* : Cartographier et analyser les bibliothèques et frameworks tiers intégrés au projet pour identifier les vulnérabilités publiques connues (CVE).
 - *Outils recommandés* : **Snyk Open Source** ou **OWASP Dependency-Check**.

4. Intégration SIEM et Rôle Stratégique de Splunk

Dans le cadre de cette architecture de sécurité, il est crucial de préciser le positionnement de **Splunk**.

Il convient de noter que Splunk n'opère pas comme un scanner de vulnérabilités actif (à l'inverse d'outils comme Nessus ou Burp Suite). Splunk est déployé en tant que solution **SIEM (Security Information and Event Management)** de supervision globale. Son intégration dans la stratégie de sécurité se déroule comme suit :

1. **Ingestion des données de vulnérabilité** : Les résultats des scans de vulnérabilités (via Nessus, Burp Suite, etc.) sont exportés et indexés dans Splunk.
2. **Centralisation des journaux** : Splunk collecte les logs d'application sécurisés (Phase 1), les logs des serveurs web, ainsi que les alertes du WAF.
3. **Corrélation et détection en temps réel** : Splunk agit comme le moteur d'analyse central. Il croise les données de configuration avec les événements réseau. Par exemple, le SIEM générera une alerte critique s'il détecte une adresse IP tentant activement d'exploiter une vulnérabilité (confirmée par les scans) sur l'un des serveurs de l'infrastructure Kechbus Ticket.